

Corpus Development Manual

I. CORPUS DEVELOPMENT

Smart contracts deployed on the Ethereum blockchain are publicly accessible, which makes them vulnerable to various types of attacks. To detect and mitigate these vulnerabilities, researchers have employed a range of deep learning and machine learning techniques. The effectiveness of these approaches, however, relies heavily on the availability of reliable, comprehensive datasets that contain smart contracts and related critical information. A large, labeled dataset is crucial not only for training machine learning models but also for evaluating the performance of other vulnerability detection tools, including those based on formal methods. While machine learning-based approaches for vulnerability detection typically either create their own datasets or rely on existing ones, the lack of a large, unbiased, real-world dataset poses a significant challenge for conducting thorough evaluations of these tools. In this section, we present the development of a uniform, labeled, large-scale dataset of smart contract vulnerabilities, with a particular focus on those identified in the “OWASP Smart Contract Top 10”. The Smart Contract Weakness Classification Registry (SWC Registry) serves as a widely recognized resource cataloging known smart contract vulnerabilities specific to the Ethereum platform. Along with it the ‘OWASP Smart Contract Top 10’ is a foundational awareness document aimed at providing Web3 developers and security teams with insights into the ten most prevalent vulnerabilities commonly found in smart contracts. Since, the SWC Registry has not been updated with new entries since 2020, the ongoing development of tools for detecting these weaknesses underscores their enduring relevance in smart contract security.

A. Data Collection

The availability of datasets focused on Solidity vulnerabilities in the literature is quite limited, with most comprising real-world smart contracts. However, a significant challenge lies in the fragmented nature of these datasets, as they are often utilized in isolated research domains and tailored to specific, narrowly-defined purposes. This fragmentation raises concerns about the fairness and representativeness of dataset selection. Moreover, while many researchers have labeled hundreds of smart contracts for tool evaluation, a considerable portion of these contracts are toy examples, characterized by significantly fewer lines of code compared to real-world decentralized application (DApp) projects.

To address this issue, we conducted an extensive review of the literature to identify the most commonly used datasets. To ensure the relevance and quality of the datasets selected, we established well-defined inclusion and exclusion criteria. These criteria were meticulously designed to align with the

objectives of our study, particularly in the context of vulnerability detection in smart contracts.

The inclusion criteria under which a dataset should be included in our study:

- It should be published at top software engineering and security conferences and journals, i.e., ASE, FSE, ICSE, ISSTA, S&P, CCS, USENIX Security, NDSS, TSE, TOSEM, etc.
- It should include Ethereum smart contracts source code that are known to contain vulnerabilities.
- The dataset should be publicly available and widely used in existing literature and research.
- The dataset should be compatible with existing tools used for smart contract analysis and preprocessing.
- The dataset should include a large number of contracts and should cover a reasonable variety of code patterns.

The exclusion criteria under which a dataset should be excluded from our study:

- Smart contracts that do not pertain to Ethereum.
- Smart contracts that do not compile or contain syntax errors should be excluded.
- Smart contracts that are private, or otherwise restricted in access due to licensing or other legal considerations should be excluded
- Duplicates, where multiple copies of the same smart contract exist in the dataset, should be excluded.

After applying our inclusion and exclusion criteria, we identified five relevant datasets for our study: SBCurated, CodeSmell, ICSE_Data, ESC, and SBWild . All smart contracts within these datasets were sourced from trusted and reputable entities in the field. Furthermore, we ensured traceability by providing, wherever possible, the URL of the source and details about the author of each contract.

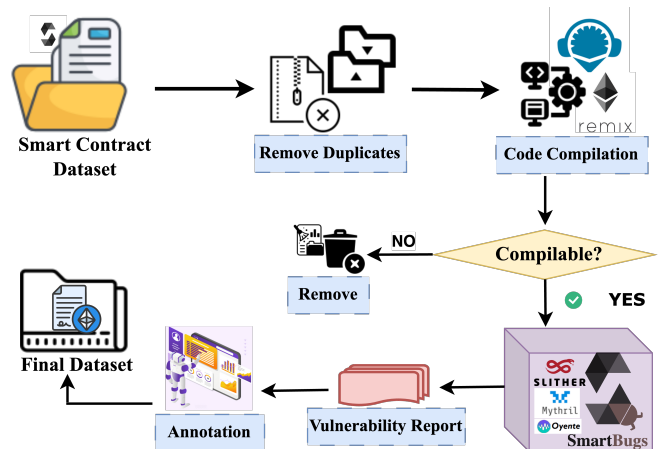


Fig. 1: Flowchart depicting the annotation process for the dataset

B. Data Preprocessing

Let $\mathcal{D} = \{C_1, C_2, \dots, C_n\}$ represent the set of all smart contracts initially collected from publicly available datasets, where each C_i is a smart contract source code. We aim to preprocess, re-annotate, and relabel the dataset $\mathcal{D}$ to create a high-quality labeled dataset $\mathcal{D}'$ that can be used for training and evaluating machine learning models for vulnerability detection. Figure 1 illustrates the process of systematically labeling and annotating the datasets to ensure uniformity. Following this, the dataset was filtered to eliminate duplicate contracts. Two contracts were identified as duplicates if their MD5 checksums matched, after preprocessing the source files by removing all spaces and comments.

a) *Removal of Duplicates*:: Define a preprocessing function $\text{preprocess}(C_i)$ that removes all spaces and comments from the source code of contract C_i . For a given contract C_i , the preprocessed code is:

$$C_i^{\text{preprocessed}} = \text{preprocess}(C_i)$$

Now, define the MD5 checksum function $\text{md5}(\cdot)$, which computes the unique hash of a given source file. Two contracts C_i and C_j are considered duplicates if:

$$\text{md5}(C_i^{\text{preprocessed}}) = \text{md5}(C_j^{\text{preprocessed}})$$

Let $\mathcal{U} \subseteq \mathcal{D}$ represent the set of unique contracts after removing duplicates. Mathematically, we can define $\mathcal{U}$ as:

$$\mathcal{U} = \{C_i \in \mathcal{D} \mid \forall C_j \in \mathcal{D}, \text{ if } i \neq j, \\ \text{md5}(C_i^{\text{preprocessed}}) \neq \text{md5}(C_j^{\text{preprocessed}})\}$$

b) *Filtering for Compilable Contracts*: From the set of unique contracts $\mathcal{U}$, we filter out the contracts that are not compilable using Remix IDE. Define the compilability function:

$$\text{compilable}(C_i) = \begin{cases} \text{True}, & \text{if } C_i \text{ compiles in Remix IDE,} \\ \text{False}, & \text{otherwise.} \end{cases}$$

Let $\mathcal{C} \subseteq \mathcal{U}$ represent the subset of contracts that are compilable:

$$\mathcal{C} = \{C_i \in \mathcal{U} \mid \text{compilable}(C_i) = \text{True}\}$$

The final filtered dataset, $\mathcal{D}_{\text{filtered}}$, is equivalent to the set of compilable contracts:

$$\mathcal{D}_{\text{filtered}} = \mathcal{C}$$

This dataset consists of unique, compilable smart contracts and serves as the foundation for further processing and analysis.

C. Annotation Manual

We considered seven vulnerability types commonly referenced in OWASP and SWC standards. Table I provides a detailed description of each vulnerability type, along with its notation and corresponding OWASP and SWC identifiers. Following the guidelines highlighted in SB and IEEE S&P, we established a consistent rule for labeling vulnerabilities.

Sl. No.	Vulnerability Type	Notation	OWASP-ID	SWC-ID	Description
1.	Reentrancy	REENT	SC01:2023	SWC-107	When a function makes an external call to another contract before updating its own state.
2.	Arithmetic (Integer Overflow and Underflow)	ART_IJOU	SC02:2023	SWC-101	When an arithmetic operation exceeds (or goes below) the maximum value (or minimum value) a data type can hold.
3.	Timestamp Dependence	TD	SC03:2023	SWC-116	When using block.timestamp for time-sensitive functions.
4.	Access Control	AC	SC04:2023	SWC-105 & SWC-106 & SWC-113	Allows unauthorized users to access or modify a contract's data or functions.
5.	Transaction Order Dependence	TOD	SC05:2023	SWC-114 & SWC-104	Exploits knowledge of pending transactions to gain an unfair advantage.
6.	Unchecked Low Level Calls	ULLC	SC10:2023	SWC-127 & SWC-115	Uses assembly instructions, such as MSTORE or assign operator and failing to properly verify the outcome of external function calls.
7.	Tx-Origin	TxO	-	-	Using the variable 'tx.origin' for authorization

TABLE I: Notation of the vulnerabilities under consideration.

Sl. No.	Tool Name	Vulnerability Type						
		REENT	ART_IJOU	TD	AC	TOD	ULLC	TxO
1.	Slither	✓	×	✓	✓	×	×	×
2.	Oyente	✓	×	✓	×	×	×	×
3.	Osiris	×	✓	×	×	×	×	×
4.	Mythril	✓	✓	✓	✓	×	✓	✓

TABLE II: Vulnerability supported by selected tools (add approach and tool version)

Table II presents an overview of which tools best identify each vulnerability type.

To determine whether a smart contract is vulnerable, we adopted a conservative approach: a contract is marked as vulnerable to a specific type if at least one tool detects it. This decision is rooted in the principle that false negatives (incorrectly indicates that vulnerabilities are absent) are far more dangerous than false positives (incorrectly indicates that vulnerabilities are present). False negatives provide a false sense of security, which can lead to severe risks, whereas false positives, though inconvenient, err on the side of caution.

The following rules were used for labeling smart contracts with specific vulnerabilities:

- 1) **Reentrancy Vulnerability**: A contract is labeled as reentrant if identified by any of the following tools: Slither, Oyente, or Mythril.
- 2) **Arithmetic Vulnerability**: A contract is labeled as having arithmetic issues if flagged by either Osiris or Mythril.
- 3) **Timestamp Dependence Vulnerability**: A contract is labeled as timestamp-dependent if detected by Slither, Oyente, or Mythril.
- 4) **Access Control Vulnerability**: A contract is flagged for access control issues if identified by either Slither or Mythril.
- 5) **Transaction Order Dependency Vulnerability**: A contract is labeled as transaction-order dependent if flagged by either Oyente or Mythril.
- 6) **Unchecked Low-Level Calls Vulnerability**: A contract is flagged for unchecked low-level calls if identified by either Slither or Mythril.
- 7) **Tx.origin Vulnerability**: A contract is labeled with a tx.origin vulnerability if flagged by either Slither or Mythril.

Let the set of vulnerabilities to be labeled be represented as:

$$\mathcal{V} = \{V_1, V_2, \dots, V_k\}$$

where each V_i corresponds to a specific type of vulnerability (e.g., reentrancy, arithmetic overflow, etc.).

Let the set of tools used for vulnerability detection be:

$$\mathcal{T} = \{T_1, T_2, \dots, T_m\}$$

where each T_i is a tool capable of detecting specific vulnerabilities in a smart contract C_j . The function $T_i(C_j)$ represents the set of vulnerabilities detected by tool T_i for contract C_j .

For each contract $C_j \in \mathcal{D}_{\text{filtered}}$, the combined set of vulnerabilities detected by all tools is given by:

$$\mathcal{T}(C_j) = \bigcup_{i=1}^m T_i(C_j)$$

where $\mathcal{T}(C_j)$ is the union of all vulnerabilities detected by the tools in $\mathcal{T}$ for contract C_j .

For each vulnerability type $V_i \in \mathcal{V}$, let $\mathcal{T}_{V_i} \subseteq \mathcal{T}$ denote the set of tools capable of detecting V_i . A contract C_j is labeled as vulnerable to V_i if at least one tool in $\mathcal{T}_{V_i}$ detects V_i . Formally:

$$V_i(C_j) = \begin{cases} 1, & \text{if } \exists T_k \in \mathcal{T}_{V_i}, T_k(C_j) \ni V_i, \\ 0, & \text{otherwise.} \end{cases}$$

where $V_i(C_j) = 1$ indicates that contract C_j is vulnerable to V_i , and $V_i(C_j) = 0$ indicates that it is not.

D. Data Annotation

The motivation for re-annotating existing datasets, such as SBCurated, Solidifi, and Codesmell, stems from the heterogeneity in how vulnerabilities are labeled within these datasets. To ensure consistency, we adhered to a standardized "Annotation Manual," as outlined in Section I-C. Additionally, these datasets vary in the number and types of vulnerabilities considered, further underscoring the need for consistent annotation. This uniform re-annotation of existing datasets allows us to fairly compare and benchmark state-of-the-art machine learning approaches in the literature.

Different tools have their way of naming and representing a vulnerability. So made a uniform label.

Construct the annotated dataset, $\mathcal{A}$, contains labels for each contract $C_j \in \mathcal{D}_{\text{filtered}}$ and each vulnerability $V_i \in \mathcal{V}$. Mathematically:

$$\mathcal{A} = \{(C_j, V_i, V_i(C_j)) \mid C_j \in \mathcal{D}_{\text{filtered}}, V_i \in \mathcal{V}\}$$

Here, $(C_j, V_i, V_i(C_j))$ represents the annotation of contract C_j for vulnerability V_i , where $V_i(C_j) = 1$ or $V_i(C_j) = 0$.